

Lab 3: Big Data Architectures

23D020 Big Data Management for Data Science

Anabel Pichardo, Cameron Armour

June 2026

A.1 Datasets and Analysis

We predict the housing price tier of a Barcelona neighbourhood in a given year. The target has three classes, low, mid, and high, defined as the per-year terciles of the average sale price per square metre. The unit of analysis is the neighbourhood-year, identified by the official neighbourhood code `Codi_Barri` and the year.

We integrate three Open Data BCN datasets, all at the neighbourhood-year grain:

Dataset	Format	Coverage	Provides	Role
Family disposable income (RFD)	CSV	2007–2017	Income index	Feature
Population and density	JSON	2010–2021	Population, density, surface	Feature
Housing sale price	JSON	2013–2017	Sale price per m ²	Target source

Table 1: Selected Datasets

All three carry `Codi_Barri` and a year, so they join exactly and need no name reconciliation. Their common period is 2013 to 2017. An inner join on neighbourhood and year over this window yields 359 rows with no missing values in the retained columns. The features are the income index, population, gross density, net density, surface, and a per-year rank of the income index. The price columns define the label, so we remove them from the features to avoid leakage.

We predict the price tier rather than an income or density tier for two reasons. With income as the label, the density data alone already separates the classes well and the price data adds no value, so the third dataset would not be justified. With density as the label, the anti-leakage rule forces us to drop population and surface as well, since the density measures are built from them, which leaves almost no usable features. Predicting the price tier keeps both income and density as informative, non-leaking predictors. Cutting the tiers per year removes the city-wide price increase across 2013 to 2017, so the label reflects a neighbourhood’s standing relative to the city in that year. The resulting classes are balanced, with 119, 120, and 120 members.

We also reviewed the unemployment, incidence, and real-estate listing datasets and did not use them. The unemployment and incidence files are truncated samples, returning about one hundred records per year and, for unemployment, a single month, with the 2015 unemployment file empty, which leaves too little signal at the neighbourhood-year grain. The listing dataset covers only 2020 and 2021, so it does not overlap the income and price series.

A.2 Architecture

The architecture is organised as a set of Spark DataFrame pipelines over three data lake zones, followed by a Spark ML and MLflow modelling pipeline. We separate these stages because each zone has a different responsibility. The Landing Zone preserves a reproducible raw snapshot, the Formatted Zone applies reusable cleaning and typing rules, and the Exploitation Zone is specific to predicting Barcelona neighbourhood price tiers. The Formatted and Exploitation zones are partitioned by year because downstream joins and model training operate over the shared 2013–2017 window.

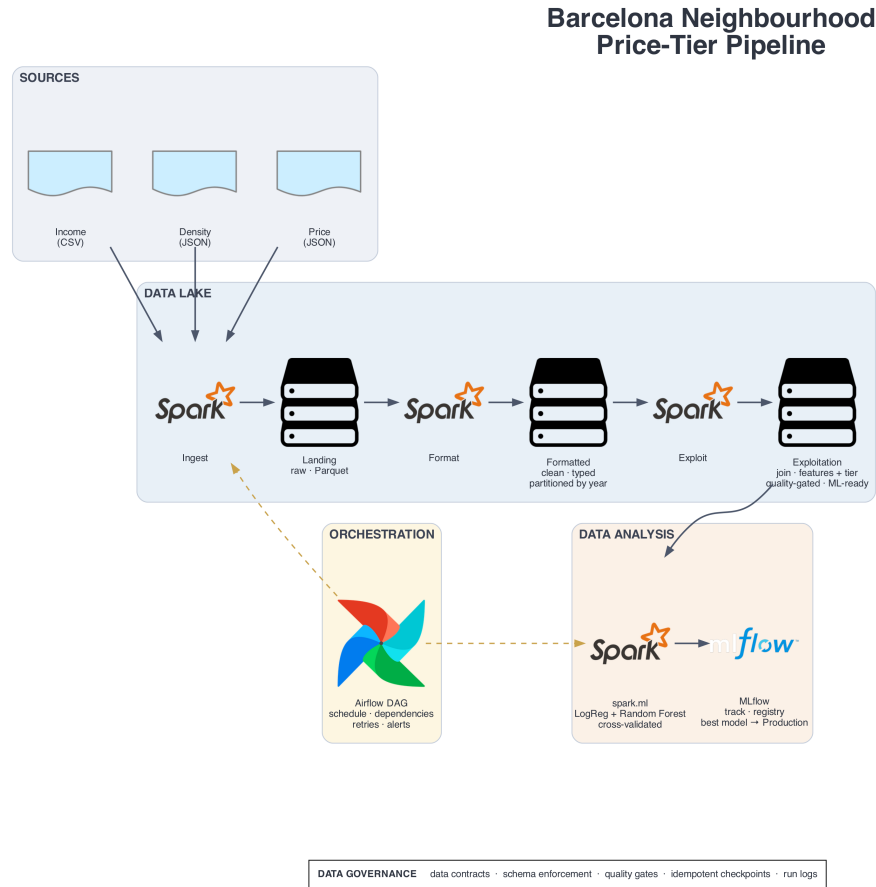


Figure 1: High-level architecture of the Barcelona neighbourhood price-tier pipeline.

Figure 1 shows the complete flow from source files to model management. The important design decision is that modelling does not read directly from the raw or formatted data. Instead, it consumes one explicit exploitation table, built at the neighbourhood-year grain. This keeps the machine-learning input stable and makes it clear where features and labels are created.

Figure 2 expands the Spark pipelines and groups related Spark operations into boxes.

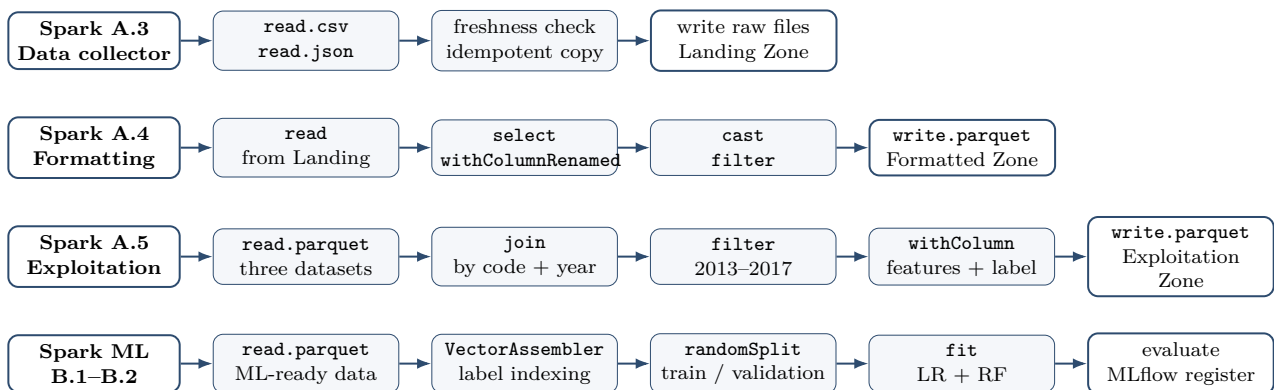


Figure 2: Spark pipelines expressed as grouped PySpark DataFrame and Spark ML operations. Grey boxes are transformations; white final boxes are materialisation, evaluation, or registration steps.

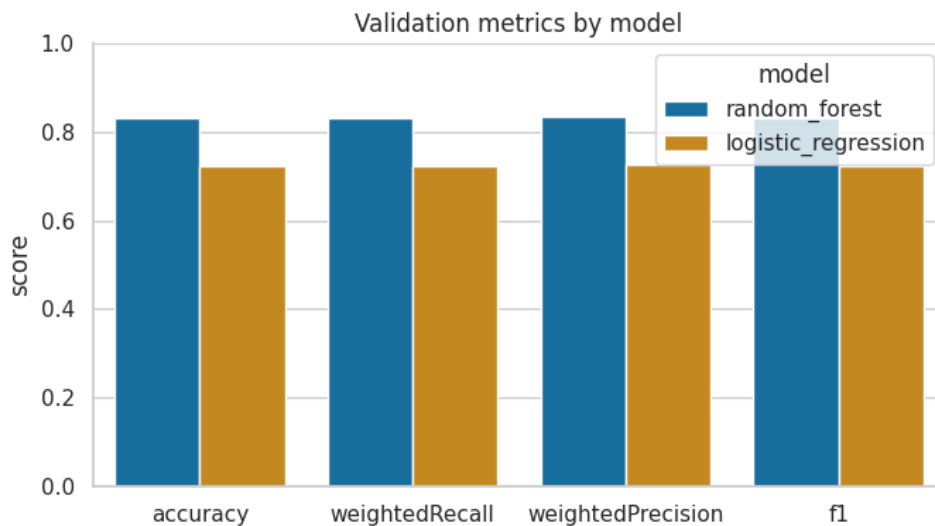
The architecture separates deterministic data preparation from learned model training. The Exploitation Zone fixes the modelling grain, joins the standardised datasets, creates the target and removes leakage-

prone price fields. The Spark ML stage then consumes this stable table to train, compare and register models. Airflow orchestrates these stages in dependency order, so each downstream task runs only after its required upstream outputs have been rebuilt. This gives the pipeline a single control layer for scheduling, parallel task execution, failure handling and reruns.

B.3 Model Training and Validation Results

The final exploitation table contains 359 neighbourhood-year rows after joining the three formatted datasets over the shared 2013–2017 period. As mentioned above, the target is balanced by construction as price per square metre is converted into three within-year tiers. This makes accuracy meaningful, while weighted recall, weighted precision and F1 are also reported to check that performance is not driven by a single class.

The model input contains only non-price features: `rfd_index`, `population`, `density_gross`, `density_net`, and `surface`. The raw price variables used to create the target are dropped before training. The training job uses a fixed seed of 42 and an 80/20 train-validation split after sorting the exploitation table, making the validation results reproducible. Each algorithm is wrapped in a Spark ML pipeline consisting of a target `StringIndexer`, a `VectorAssembler`, and the classifier.



The validation results show that Random Forest performs consistently better than Logistic Regression across all reported metrics. Random Forest scores about 0.83 for accuracy, weighted recall, weighted precision and F1, while Logistic Regression is around 0.72. Since the target classes are balanced by construction, accuracy is a meaningful headline metric, but the similar improvements in precision, recall and F1 suggest that the Random Forest advantage is not limited to one class or one type of error.

This result suggests that the relationship between neighbourhood income, population, density, surface and price tier is not purely linear. Logistic Regression can only model linear class boundaries, whereas Random Forest can capture thresholds and interactions between variables, such as cases where income has a different relationship with price tier depending on density or neighbourhood size. However, the dataset is small, with 359 neighbourhood-year observations, so the results should still be interpreted cautiously.

MLflow is used for model management. Each run logs the model type, selected hyperparameters, feature list, validation metrics, per-class precision/recall and the fitted Spark pipeline artifact. The training job ranks runs by validation accuracy, registers the best model under `barcelona_price_tier`, promotes it to production, reloads the registered model, and evaluates it again. The reloaded model obtains the same validation accuracy of 0.83, confirming that the registered artifact can be recovered and used for prediction.

Assumptions and Justifications

Data grain and joining. We use neighbourhood-year as the common grain because it is the most detailed level at which the selected income, density and price datasets can be aligned consistently. This avoids aggregating to district level, which would lose neighbourhood variation, while also avoiding a finer grain that some sources cannot support. We use an inner join over the shared 2013–2017 window so that every modelling row is complete and no imputation is needed. The trade-off is that the final table is smaller, but the retained rows are directly comparable across all three sources.

A.3–A.5 Data engineering. The zones are separated so that each stage has a clear responsibility. The Landing Zone stores refreshed source snapshots as Parquet with ingestion metadata and minimal schema projection, giving a reproducible copy of the inputs while avoiding early analysis-specific cleaning. Explicit Spark schemas are used instead of automatic inference because several fields, such as the income index and price values, need consistent types across runs.

Leakage and validation checks. Price fields are needed to construct `price_tier`, but they are removed from the feature matrix before training so that the model cannot directly recover the answer from the target source. This matters because otherwise the validation accuracy would overstate the model’s real predictive value. The exploitation contract checks that the final table is non-empty, has the expected columns, contains no nulls in key features or labels, has unique `(Codi_Barri, year)` rows and includes all three target classes. These checks do not prove that the model is optimal, but they reduce the risk of training on a broken or inconsistent dataset.

B.1–B.2 Modelling. We use a fixed seed so that the train-validation split and model comparison are reproducible. The 80/20 split keeps most observations for training while reserving a held-out validation set for comparison. Three-fold cross-validation is applied on the training set to tune hyperparameters without using the validation set directly. This is appropriate for a small lab dataset, but the random split may still be optimistic because neighbouring years for the same area can appear in both train and validation. MLflow records parameters, metrics, feature lists and fitted Spark models, and the best run is registered as the production model so that the chosen model can be reloaded without hardcoding a run-specific path.

C.1–C.2 Orchestration. The Airflow DAG runs daily at midnight with `catchup=False`. Daily scheduling is a conservative choice for the lab reflecting our lack of detailed knowledge of the source update frequency. If the real update timing was known, the schedule could be changed to match it more closely. The DAG starts with a preflight check, then runs ingestion, formatting, exploitation and modelling in dependency order so that downstream stages only run after their inputs have been rebuilt. Each task has two retries, a five-minute retry delay and a 30-minute timeout to handle transient failures. Failure and completion alerts are implemented through Airflow callbacks that emit explicit alert messages in the task logs. We initially implemented email alerts, but removed these to avoid requiring configuration for marking. In a production deployment, the same callbacks could be connected to email or another external notification service.

Limitations. The joined dataset is small because the common period is limited by the price series, so the validation metrics should be interpreted cautiously. The model predicts relative price tier rather than exact price per square metre, which fits the classification task and removes the city-wide price trend, but it also loses within-tier variation. Future work could add newer socioeconomic data, extend the feature set with other neighbourhood-level variables, and use time-based validation by training on earlier years and validating on later years.